# 非エンジニア向け

## Sources 機能 取扱説明書

### １．この機能でできること

ワークフローに入力するデータを管理する機能です。

**できること：**

- ファイルや表データをワークフローで使えるように登録
- 一度登録したデータを繰り返し使える
- 複数の処理結果をまとめて新しいデータセットを作れる
- 処理結果を保存して別の処理で再利用できる

**2つの使い方：**

- **ファイルモード**: 複数のファイル（PDFなど）をまとめて管理
- **クエリモード**: Excelのような表データ + 関連ファイルを管理

**2種類のタイプ：**

- **一時的（Ephemeral）**: 一度だけ使う使い捨てデータ
- **永続的（Persistent）**: 保存して何度も使えるデータ

**必要なもの：**

- ファイルモード: ZIPファイル（中に複数のファイル）
- クエリモード: CSVファイル（+ 関連ファイルのZIP）

### ２．基本の使い方

#### 使用例1: 一時的にファイルをまとめて処理

**やりたいこと：** 今月の契約書PDFをまとめて処理（一度だけ）

```
APIを呼び出し
POST /projects/{project_id}/sources/ephemeral/file

必要な情報:
```

```
- 名前: "契約書バッチ処理"
- 説明: "2024年1月の契約書"
- ファイル: contract_001.pdf, contract_002.pdf など

結果:
→ ID（例: uuid-xxxx-xxxx）が発行される

ワークフローで使用:
→ このIDを指定してワークフロー実行
```

## 使用例2: ファイルを保存して繰り返し使う

**やりたいこと：** 製品マニュアルをずっと使えるように保存

```
APIを呼び出し
POST /projects/{project_id}/sources/persistent/file

必要な情報:
- ZIPファイル: 全製品のマニュアルPDF（50個入り）
- 名前: "製品マニュアル集"
- 説明: "全製品のマニュアルPDF"

結果:
→ ID（例: uuid-yyyy-yyyy）が発行される
→ このIDで何度でも使える
```

## 使用例3: 表データを一時的に処理

**やりたいこと：** 顧客リストのCSVを一度だけ処理

```
APIを呼び出し
POST /projects/{project_id}/sources/ephemeral/query

必要な情報:
- 名前: "顧客リスト処理"
- CSVデータ: 顧客ID、名前、メールアドレスなど
- データの型定義:
  - customer_id は数字
  - name は文字
```

```
  - email は文字

結果:
→ ID（例: uuid-zzzz-zzzz）が発行される
```

## 使用例4: 表データを保存して繰り返し使う

**やりたいこと：** 取引先マスタを登録して何度も使う

```
APIを呼び出し
POST /projects/{project_id}/sources/persistent/query

必要な情報:
- CSVファイル: 取引先の情報
- 関連ファイルのZIP: 契約書PDFなど（オプション）
- 名前: "取引先マスタ"
- データの型定義:
  - company_id は数字
  - company_name は文字
  - contract_file は関連ファイルへの参照

結果:
→ IDが発行される
→ 後で必要な時にCSVでダウンロードできる
```

## 使用例5: 複数の処理結果をまとめる

**やりたいこと：** 先月と今月の処理結果を1つにまとめる

```
APIを呼び出し
POST /projects/{project_id}/sources/persistent/from-multipl
e-sources

必要な情報:
- 名前: "統合データセット"
- まとめたいデータ:
  - 先月の処理結果（ID: flow-run-id-1）
  - 保存済みデータ（ID: source-set-id-1）
  - 今月の処理結果（ID: flow-run-id-2）
```

```
処理の流れ:
1. それぞれのデータを取得
2. 共通する列（項目）を見つける
3. データの型が合うか確認
4. 縦に並べて1つにまとめる
5. 新しいデータセットとして保存

結果:
→ 新しいIDが発行される
→ まとめる前のデータ情報も記録される
```

# ３．使い方のコツと注意点

## よく使うパターン

### 1. データを保存して段階的に処理

**流れ：** ファイル保存 → 処理1 → 結果保存 → 処理2 → 結果保存

- **やること：** 1つのデータを何段階かに分けて処理
- **使う場面：** 複雑な処理を分けて実行したい時

### 2. 一時データでサッと処理

**流れ：** CSVアップロード → 処理 → 完了

- **やること：** 使い捨てデータの処理
- **使う場面：** 毎回違うデータを処理する時

### 3. 複数の処理結果を統合

**流れ：** 処理A → 処理B → 処理C → 全部まとめる

- **やること：** バラバラの結果を1つにまとめる
- **使う場面：** 複数の処理結果を一括で分析したい時

### 4. データとファイルをセットで管理

**流れ：** CSV + 関連PDFをアップロード → 処理

- **やること：** 表データと実ファイルを一緒に扱う

- **使う場面：** 顧客情報と契約書PDFなど、データとファイルがセットの時

## 気をつけること

### 📁 どのモードを使う？

- **ファイルモード**: PDFやWordなど、個別のファイルをたくさん処理する時
- **クエリモード**: Excelみたいな表データを処理する時
- **一時的**: 一度きりの処理
- **永続的**: 何度も使う、結果を保存したい

### 📋 表データの型を定義

クエリモードでは、各列のデータ型を指定します：

- **数字（int）**: 1, 2, 3...
- **小数（float）**: 1.5, 2.3...
- **文字（string）**: 名前、住所など
- **ファイル参照（fileref）**: 添付ファイルのパス

### 📎 ファイル参照の使い方

- 表データの中で「このファイルを使う」と指定できる
- 例：CSV内に「添付/契約書001.pdf」と書く
- 実際のファイルは別のZIPファイルに入れる

### 🔗 複数データをまとめる時の注意

- **同じ列がないとまとめられない**
  - 例：データAに「名前」「年齢」、データBに「名前」「住所」
  - → 共通する「名前」だけが残る
- **データの型が合わないとエラー**
  - 例：データAの「年齢」が数字、データBの「年齢」が文字
  - → エラーになる
  - ただし、数字（int）と小数（float）はOK

### 💾 保存先はS3

- データは自動的に安全な場所（S3）に保存されます
- 削除すると元に戻せないので注意

## ⏱️ 一時データの有効期限

- 一時的（Ephemeral）なデータは自動で削除されます
- 保存が必要なら永続的（Persistent）を使いましょう

## 🔒 データは変更できない

- 一度作ったデータセットは変更できません
- 変更が必要な場合は新しく作り直します
- これで「いつ処理したか」が確実に分かります

## 🏷️ 分かりやすい名前をつける

- 名前: 簡潔で分かりやすく（例：「2024年1月契約書」）
- 説明: 詳しく書く（例：「2024年1月の新規契約書50件」）
- 後で探しやすくなります

## ✅ テストは一時データで

- 試しに動かす時は「一時的」を使う
- うまく動いたら「永続的」で本番実行
- コストと容量の節約になります

## 🔄 エラーが出たら

- **型が合わない** → 表の定義を確認
- **ファイルが見つからない** → ファイルパスを確認
- **まとめられない** → 共通の列があるか確認
- エラーメッセージに詳しい説明が出ます